

FOR JAVA DEVELOPERS MOVING TO NESTJS

Bob saw Alice.

Spring Boot to NestJS, episode 24: caching with Redis

A cache is a copy with a key, a lifetime and a fill policy. Get any of the three wrong and it shows.

Bob was served Alice's cached account on both stacks: Nest keyed by URL, Spring's method took no arguments. The verified user in the key fixed it.

No time to live is the default on both Redis setups. A write left the old copy until it was evicted.

Twenty concurrent requests on a cold key loaded it twenty times. `sync = true` helped only where the cache supports it.

In scope

- `@Cacheable`, `@CacheEvict`, `sync = true`
- `CacheInterceptor`, `trackBy`, `CACHE_MANAGER`
- Redis with Spring Data Redis and `@keyv/redis`
- Keys, TTL, stampedes, misses

Not in scope

- HTTP caching headers and CDNs
- Serialization formats

At the time of recording the Nest docs describe `@nestjs/cache-manager`.

01 The measurements

GET /me AS ALICE, THEN BOB	Bob got Alice's on both; user in the key fixed it
UPDATE, THEN READ	stale on both until evicted
NO TTL, WAIT 1.2 s	1 load on both: never expires
20 AT ONCE, COLD KEY	plain 20; sync: memory 1, Redis default 20, locking 1
NEST, SHARING THE IN-FLIGHT LOAD	1 per process; 2 across two replicas
20 GETs FOR A MISSING PAYMENT	Spring 1 (null cached); Nest interceptor 20

Redis unless stated.



02 Put the user in the key

CacheInterceptor keys by URL; a no-argument @Cacheable shares one key. Anything personal needs the verified user in the key.

AccountService.java

```
@Cacheable(cacheNames = "accounts", key = "#user")
public Account meFor(String user) {
    return new Account(user, "alice".equals(user) ? 1200 : 40);
}
```

per-user-cache.interceptor.ts

```
protected trackBy(context: ExecutionContext): string | undefined {
    const url = super.trackBy(context);
    const user = context.switchToHttp().getRequest<{ headers: Record<string, string> }>
().headers['x-user'];
    return url && user ? `${url}:${user}` : undefined;
}
```

with the user in the key, Bob got his own balance on both stacks

Run on a real machine, 2026-09-30, by scripts/verify-cache.sh in the course repository.

03 One load for many

`sync = true` asks the cache to coordinate the load. On Redis it did so only with the locking cache writer. In Nest, share the in-flight promise, per process.

CacheConfig.java

```
RedisCacheManager.builder(RedisCacheWriter.lockingRedisCacheWriter(connections))
    .cacheDefaults(RedisCacheConfiguration.defaultCacheConfig().entryTtl(ttl))
    .build();
```

payments.reader.ts

```
let load = this.inFlight.get(key);
if (!load) {
  load = this.payments.find(id).then(async (payment) => {
    await this.cache.set(key, payment);
    return payment;
  }).finally(() => this.inFlight.delete(key));
  this.inFlight.set(key, load);
}
```

two replicas, one Redis: locking writer 1 load; Nest in-flight sharing 2

Run on a real machine, 2026-09-30, by `scripts/verify-cache.sh` in the course repository.

Before you trust a cache

1. Is the verified user in the key of anything personal?
2. What evicts the entry when the data changes?
3. Is there a time to live at all?
4. What happens when twenty requests miss at once?
5. Is a miss cached, and for how long?
6. Memory is per process; Redis is shared

Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- Every measurement, one command: scripts/verify-cache.sh**
<https://github.com/code-with-sam-dev/spring-to-node>
- NestJS caching**
<https://docs.nestjs.com/techniques/caching>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Redis is a trademark of Redis Ltd. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.